Project: NotebookLM/Gemini Notebook Clone

Project Goal

Build a full-stack AI application that allows users to create multiple notebooks, upload or link source material, ask questions about their sources using **Retrieval-Augmented Generation (RAG)**, and generate useful artifacts from the notebook content.

The goal is to demonstrate that you can combine **AI/RAG techniques, backend services, a user interface, data storage, and cloud deployment** into one working application.

---

1. Required Features

Your application must support the following:

Notebook Management

Users should be able to:

- Create a notebook
- Rename a notebook
- Delete a notebook
- Switch between notebooks
- Store sources and conversations separately for each notebook

Source Ingestion

Users should be able to add:

- PDF files
- PPTX files
- TXT files
- Web URLs

Your application should:

1. Extract the text
2. Split the text into chunks
3. Generate embeddings
4. Store the embeddings in a vector database
5. Associate the content with the correct notebook

You may use tools such as **ChromaDB, FAISS, Hugging Face embeddings, or another appropriate vector database/embedding solution**.

---

2. RAG Chat

Each notebook should have a chat interface that allows users to ask questions about its sources.

The RAG pipeline should:

1. Receive the user's question
2. Retrieve relevant chunks
3. Provide the retrieved information to an LLM
4. Generate an answer
5. Display citations or source references

The answer should be based on the notebook's uploaded/linked sources.

Your application should also store the conversation so that users can return to a notebook and view previous messages.

---

3. Artifact Generation

Your application must allow users to generate artifacts from notebook content.

Required

Report

- Generate a report based on the notebook sources
- Save the report as a `.md` file
- Allow the user to view or download it

Quiz

- Generate a quiz based on the notebook sources
- Save the quiz as a `.md` file
- Include an answer key
- Allow the user to view or download it

Optional

You may implement additional artifact types such as:

- Podcast transcript
- Podcast audio
- Flashcards
- Mind maps
- Other useful study materials

---

4. Storage

Your application should organize data by notebook.

The important requirement is that:

- Each notebook has a unique ID
- Sources belong to a specific notebook
- Chat history belongs to a specific notebook
- Generated artifacts belong to a specific notebook
- Data can be retrieved when the application is restarted

If the deployment environment has limitations on persistent storage, **document those limitations rather than trying to solve them with unnecessary infrastructure.**

---

5. User Interface

You may use **Gradio** or another appropriate framework.

The interface should provide access to:

- Notebook management
- Source upload
- URL ingestion
- Chat
- Citations
- Artifact generation
- Artifact viewing/downloading
- Error messages when something goes wrong

The interface does not need to be highly polished. Focus on making the application **clear, functional, and easy to use**.

---

6. RAG Evaluation

An important part of the project is evaluating your RAG system.

Implement and compare **at least two retrieval approaches**.

For example:

- Basic vector similarity search
- Vector search with reranking
- Different chunking strategies
- Different embedding models
- Hybrid search

You should explain:

- What techniques you tested
- What documents/chunks were retrieved
- How the techniques affected answer quality
- Response time
- Which approach you selected and why

You do **not** need to build a highly sophisticated evaluation framework. A small set of representative questions is sufficient.

For example:

Question	Method	Retrieved Chunks	Response Time	Answer Quality
Question 1	Vector Search	4	1.8 sec	Good
Question 1	Reranking	4	2.4 sec	Better

Discuss the tradeoffs you observed.

---

7. Cloud Deployment

Deploy your application to a cloud hosting platform. Hugging Face Spaces is one option, but you may use another comparable platform such as AWS, Render, Railway, or Google Cloud Run. Choose a platform that supports the requirements of your application. If you choose to use Google Cloud Run, there is a helpful resource provided in the assignment instructions on Canvas!

The deployed application should:

- Start successfully
- Allow users to interact with the application
- Include all required dependencies
- Use environment variables/secrets for API keys
- Not contain API keys or other secrets in the repository

Your repository should include a README explaining:

- What the application does
- How to run it locally
- Required environment variables
- How the application is structured
- How data is stored
- How to use the application

---

8. GitHub Actions / CI/CD

Set up a GitHub Actions workflow that automatically deploys changes to your cloud deployment platform.

The expected workflow is:

Make code changes

↓

Push to GitHub

↓

GitHub Actions

↓

Deploy

↓

Updated application

API tokens/secrets must be stored using **GitHub Secrets** rather than being committed to the repository.

---

9. Deployment Demonstration

Include a short **1–2 minute demonstration** showing that your application is successfully deployed.

The demonstration should show:

1. Your live cloud deployment platform
2. Creating or selecting a notebook
3. Adding a source
4. Asking a RAG question
5. Showing a citation
6. Generating an artifact
7. Your GitHub Actions workflow successfully deploying the application

The demonstration may be a short screen recording.

---

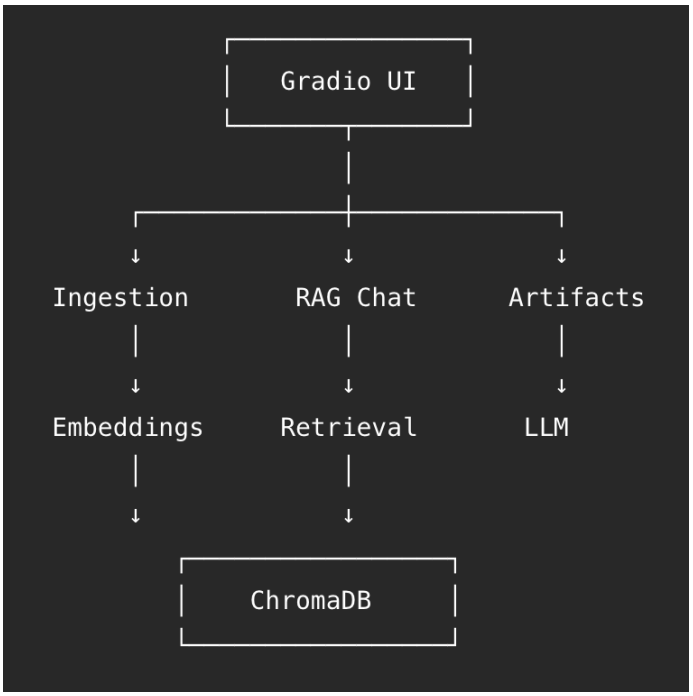
10. Architecture Documentation

Submit a short architecture document containing:

Architecture Diagram

Show the major components of your application.

For example:



Also briefly explain:

- Major modules
- Data flow
- Notebook storage
- RAG pipeline
- Artifact generation
- Deployment architecture

---

11. Specifications Before Implementation — Optional

Before beginning implementation, try creating a short specification for your application.

Your specification could define:

- Application requirements
- Major components
- Data flow
- Notebook data model
- RAG pipeline
- API/model dependencies
- Deployment approach

You may use AI coding tools such as **Claude Code, Codex, Cursor, Antigravity, or similar tools**.

However, you should first establish the application design/specification before generating the implementation.

---

12. Advanced Features — Optional

The following features are optional and can be implemented for additional functionality:

- YouTube sources
- CSV/tabular data
- Audio sources
- Flashcards
- Mind maps
- Multiple podcast speakers
- Custom prompts for artifact generation
- Enable/disable individual sources
- Additional retrieval techniques
- Additional artifact types

Do not attempt these features if they prevent you from completing the required functionality.

---

13. Deliverables

Submit:

1. GitHub Repository

Your repository should contain:

- Source code
- README
- Requirements/dependencies
- GitHub Actions workflow
- `.gitignore`
- Architecture documentation

2. Cloud Deployment Platform

Submit the URL to your deployed application.

3. RAG Evaluation

Include your comparison of the two retrieval approaches, including:

- Test questions
- Retrieved chunks
- Response times
- Answer quality
- Final conclusions

4. Deployment Demonstration

Submit a 1–2 minute screen recording demonstrating the deployed application and successful CI/CD deployment.

---

14. AI Use

You have free reign to use AI Code generation tools to help build this system.

---

15. Submission Checklist

Use the following checklist to verify you have completed all the requirements:

Application

- ☐ Application runs locally
- ☐ Application runs on your chosen cloud platform
- ☐ Multiple notebooks can be created
- ☐ Notebooks can be switched
- ☐ Notebooks can be renamed/deleted
- ☐ PDF ingestion works
- ☐ PPTX ingestion works
- ☐ TXT ingestion works
- ☐ URL ingestion works
- ☐ RAG questions can be asked
- ☐ Answers include citations/source references
- ☐ Chat history is stored
- ☐ Reports can be generated
- ☐ Quizzes can be generated
- ☐ Quiz answer keys are included
- ☐ Artifacts can be viewed/downloaded
- ☐ Appropriate error handling is implemented

Cloud Deployment Platform

- ☐ Space builds and starts successfully
- ☐ Required dependencies are included
- ☐ API keys are stored as secrets
- ☐ No API keys are committed to GitHub
- ☐ README explains the application
- ☐ Storage behavior/limitations are documented
- ☐ Deployment demonstration is included
- ☐ Live application is shown in the demonstration
- ☐ Successful GitHub Actions deployment is shown

GitHub

- ☐ Repository is accessible

- [ ] Source code is committed
- [ ] `.gitignore` is included
- [ ] GitHub Actions workflow is included
- [ ] Pushes trigger deployment
- [ ] Deployment completes successfully

Documentation

- [ ] Architecture diagram is included
- [ ] Major modules are explained
- [ ] Data flow is documented
- [ ] Storage approach is explained
- [ ] Two RAG approaches are compared
- [ ] Retrieved chunks are shown
- [ ] Response times are measured
- [ ] Tradeoffs are discussed
- [ ] Cloud deployment platform URL is provided
- [ ] GitHub URL is provided